Aspis: End-to-End Formal Verification of a Transparent Private Spend on Solana

Dominic Barker

25 August 2026

Abstract

We present Aspis, a transparent, computationally hiding private-spend system whose verifier ran on Solana mainnet. To our knowledge, following a public-evidence search completed on 24 August 2026, this is the first publicly evidenced Solana mainnet transaction in which a program directly verifies a trusted-setup-free private-spend proof and atomically records both its nullifier and the resulting pool state. The transaction verified a 75,358-byte proof in 1,334,452 compute units.

Our main result is an end-to-end formal verification of the deployed proof-checking path. From every successful translated Rust call, Lean 4 derives the public statement, byte-level Fiat–Shamir transcript, six work checks, eighteen distinct queries, five authenticated opening sections, four low-degree folds, final polynomial, complete algebraic relation, and both terminal accumulator equalities. Every fact comes from the same execution, and the theorem is universal over successful calls.

The mathematical development constructs a spend witness from the trace; proves the exact Mersenne-field domains, encoders, capped-query law, coherent four-fold candidate chain, and candidate-weight fold duality; and connects these results to theft, replay, state, and a finite failure decomposition. Published circle-decoding and Fiat–Shamir results give a work-normalized protocol contribution of at most $0.7 \cdot 2^{-100}$. With the stated external-event budget, the combined conditional bound is 2^{-100}, with raw post-grinding accounting reported separately. A clean 331-module replay, pinned translation inputs, a byte-reproducible program, and the finalized execution archive accompany the proof. The live-statement identification and causal probability lift remain explicit composition interfaces.

1 Introduction

A finalized Solana mainnet transaction verified a 75,358-byte transparent proof of a private spend, recorded the public nullifier, and replaced the pool root in one atomic state transition. The verifier used 1,334,452 of the transaction's 1,356,912 compute units. Its commitments are hash based and require no trusted setup. To our knowledge, following a public-evidence search completed on 24 August 2026, this is the first publicly evidenced Solana mainnet transaction to combine direct verification of a transparent, computationally hiding private-spend proof with the corresponding nullifier and pool-state update in the same transaction. The comparison covers publicly indexed papers, repositories, deployments, and patents available by the cutoff date. The proof account was uploaded and sealed beforehand; the reported transaction performed proof verification and the atomic spend-state update. Section 11 gives the closest public systems and exact comparison.

This paper presents that result together with a formal argument reaching from the deployed verifier to its cryptographic meaning. The central theorem begins with an arbitrary successful call to the deployed Rust proof checker, after functional translation into Lean 4. From that call alone it

derives the parsed statement, every byte absorbed by Fiat–Shamir, all six work checks, the eighteen distinct queries, five authenticated opening sections, four low-degree folds, the final polynomial, all algebraic claims and relation messages, and both final accumulator equalities. These values belong to one execution. The theorem therefore rules out a real composition error: proving the transcript for one accepted run and the openings or algebra for another does not establish anything about either run.

The span of this theorem is the principal formal contribution. Formal cryptographic developments commonly prove an abstract protocol or a collection of executable components. Here the accepted-execution theorem crosses the parser, transcript, authenticated data, low-degree test, algebraic relation, and terminal arithmetic of a deployed blockchain verifier. It is paired with reproducible evidence connecting the reviewed Rust source to the program bytes and those bytes to the finalized transaction. The result is a source-to-security argument with a concrete chain execution at its far end.

1.1 The mathematical argument

The proof system establishes a one-input, one-output spend. Its public statement contains the old and new pool roots, a one-time nullifier, the output-note commitment, asset and fee data, and the deployment domain. The witness opens the input and output notes, proves ownership and integer value conservation, and supplies a depth-twenty path that replaces the input leaf by the output commitment. We formalize this relation directly and prove that a valid authenticated algebraic trace constructs such a witness.

The low-degree argument is developed for the exact verifier rather than an asymptotic profile. Lean proves the order of the chosen generator in $\mathbb{F}_{2^{31}-1}$, distinctness and ordering of the circle and line domains, and the evaluation identities of all four encoders. The initial decoder list has size at most 240 under the published circle-decoding result. Outside the counted challenge events, one member of that list follows a single coefficient chain

$$1024 \longrightarrow 256 \longrightarrow 64 \longrightarrow 16 \longrightarrow 4$$

through the four radix-four folds and reaches the polynomial published by the prover. A separate causal argument fixes each exceptional challenge set from the transcript prefix available before that challenge is sampled.

The query calculation also follows the implemented sampler exactly. It retains the first eighteen distinct positions found in at most sixty-four draws. Conditional on success, the output is uniform over ordered injections. Thus, for a bad set of size A in a domain of size N, the exact miss probability is

$$\frac{(A)_{18}}{(N)_{18}} = \prod_{j=0}^{17} \frac{A-j}{N-j}.$$

Here $(m)_k = m(m-1)\cdots(m-k+1)$ is the falling factorial. The same expression bounds the unconditioned event because sampler exhaustion rejects. Finally, the relation proof shows algebraically that folding the candidate and folding the verifier's dual weights preserve their dot product. This carries the extracted polynomial into the spend constraints rather than leaving low degree and statement validity as separate claims.

The formal results have the following logical shape. For one translated execution ρ,

$$\mathsf{Accept}(\rho) \Longrightarrow \mathsf{Evidence}(\rho), \tag{1}$$

Table 1: Evidence supporting the end-to-end result.

Artifact	Established fact	Reproduction evidence
Lean development	Spend mathematics, functional correctness, failure decomposition, and the accepted-execution theorem	Clean replay of the complete 331-module dependency closure
Rust-to-Lean translation	Control flow and values of the successful proof-checking path	Pinned translator revision, generated Lean, and source manifests
Build record	Identity between recorded source inputs and deployed program bytes	Deterministic build command, toolchain record, and byte hashes
Mainnet record	Direct proof verification and atomic nullifier and pool update	Signed transaction, account snapshots, proof bytes, and verification script

where $\mathsf{Evidence}(\rho)$ contains the full transcript, authentication, folding, and relation data from ρ. In the security experiment, Lean separately proves the pointwise inclusion

$$A_{\text{false}} \subseteq C \cup \bigcup_{i=1}^{18} E_i, \tag{2}$$

where C contains the explicit protocol events and the E_i are named primitive, credential, implementation, and runtime events. Two displayed interfaces connect these endpoints: fieldwise agreement between the live Rust statement and the abstract statement, and a causal embedding of the deterministic accepted-call classification into the probability experiment.

1.2 End-to-end formal evidence

We use *end-to-end* for a theorem and artifact chain in which every change of representation is exposed to review. The evidence has five parts:

1. **Mathematical specification.** Lean definitions state the private-spend relation, concrete domains and encoders, transcript, authenticated openings, low-degree test, algebraic relation, security experiment, and state transition.

2. **Implementation proof.** The deployed Rust proof-checking path is functionally translated into Lean. Refinement theorems connect its control flow and returned values to the mathematical definitions.

3. **Accepted-execution composition.** One universal theorem derives every acceptance-critical fact, including both terminal dot products, from one successful translated call.

4. **Program identity.** Pinned source, dependencies, compiler tools, and a deterministic build reproduce the deployed program byte for byte.

5. **Mainnet evidence.** The archived transaction binds the program, proof, statement, compute usage, nullifier, and old and new pool states.

Table 1 records the evidence supplied for each layer. The manuscript uses mathematical names; the artifact guide maps them to exact Lean declarations, files, commands, revisions, and hashes.

The accepted-execution theorem is universal over successful calls, rather than a replay theorem for the archived proof. It reconstructs the public input digest and transcript, verifies six sequential

work conditions, obtains eighteen distinct query positions, authenticates five opening sections, checks four low-degree rounds and the final polynomial, decodes seventy-six algebraic claims and fifty-eight relation fields, and derives the terminal equalities of twelve-component and four-component accumulators. Its conclusion depends on the derived mathematical evidence, including the values consumed before the last helper returns.

1.3 Security statement

All false-acceptance events live in one finite probability space. The coverage theorem in Equation (2) is pointwise, so its probability bound uses a union bound without an independence hypothesis. For the evaluated parameters, Lean checks that the work-normalized protocol contribution satisfies

$$B_{\text{protocol}} \leq 0.7 \cdot 2^{-100}.$$

Published circle-decoding and Fiat–Shamir theorems enter through explicit interfaces whose concrete parameter hypotheses are checked in Lean. If the remaining primitive and credential events contribute at most $0.3 \cdot 2^{-100}$, and the stated deterministic correspondences hold, the combined accepted-false probability is at most 2^{-100}. A separate raw resource bound gives no security credit for grinding work that has already been completed; its dominant explicit term is approximately 2^{-70}.

1.4 Contributions

The paper makes four contributions:

- The first publicly evidenced Solana mainnet result, to our knowledge at the stated cutoff, that directly verifies a transparent, computationally hiding private-spend proof and atomically records its nullifier and new pool state within the transaction compute limit.
- An end-to-end Lean theorem for every successful translated execution of the deployed proof-checking path, covering the full chain from proof bytes and transcript operations through authenticated openings, low-degree folds, the algebraic relation, and both final accumulators.
- A complete mathematical treatment of the evaluated verifier: private-spend extraction, exact field and encoder results, coherent four-fold extraction, exact capped-query sampling, fold duality, theft and replay reductions, and the finite security experiment with checked concrete arithmetic.
- A reproducible evidence package connecting the theorem to pinned translation inputs, byte-identical program reproduction, proof and statement bytes, and a finalized mainnet state transition.

1.5 Organization

Section 2 defines the spend relation and proof parameters. Section 3 develops the checked mathematics, followed by the proof method and implementation theorem in Sections 4 and 5. Section 6 gives the single security experiment and numerical bounds. Sections 7 and 8 cover theft, state, and hiding, and Section 9 presents the formal replay, program reproduction, and mainnet evidence. Assumptions and related work follow; the appendices give proof details and the independent reproduction checklist.

2 Background and construction

2.1 Notation

Let $\mathbb{F}_{2^{31}-1}$ be the Mersenne prime field used for trace values, and let $\mathbb{K}$ be its degree-four extension used for verifier challenges. We write H for a domain-separated hash function and use distinct domains for note commitments, nullifiers, Merkle nodes, and transcript operations. A *transparent* proof uses public hash-based commitments and therefore requires no structured reference string or ceremony.

The proof system follows the interactive-oracle-proof view underlying STARKs [7, 9]. The prover commits to encoded execution data, the verifier derives public challenges from the transcript, and a FRI-style test checks proximity to a low-degree codeword [8]. Here *FRI* refers to the low-degree test: the verifier queries a few committed positions and repeatedly folds the alleged codeword until only a small polynomial remains.

2.2 Private-spend relation

The public statement is

$$x = (P, s, R_{\text{old}}, N, C_{\text{out}}, R_{\text{new}}, a, f, d),$$

where P identifies the pool, s is its sequence number, R_{old} and R_{new} are the old and new note-tree roots, N is the input nullifier, C_{out} is the output-note commitment, a is the asset identifier, f is the public fee, and d binds the statement to a deployment domain. A nullifier is a public, one-time tag derived from the input note's secret opening. Publishing it prevents a second successful spend without revealing which commitment in the note tree was consumed.

The private witness contains the input-note opening and ownership credential, the input note's depth-twenty Merkle path, and the opening data for the output note. With input and output values v_{in} and v_{out}, the relation enforces

$$v_{\text{in}} = v_{\text{out}} + f, \qquad 0 \le v_{\text{in}}, v_{\text{out}}, f < 2^{30}. \tag{3}$$

The range conditions make the field equation an integer conservation law, ruling out modular wraparound. Both notes use asset a. The input credential determines its owner and nullifier, while each note commitment binds the owner, value, asset, and note randomness.

The input path reconstructs R_{old}. Replacing its leaf by the output commitment at the same depth-twenty position, with the same direction bits and siblings, reconstructs R_{new}. Thus the evaluated statement is a one-input, one-output replacement in a fixed-position note tree.

Note commitments, nullifiers, and trace Merkle nodes use Poseidon2, an arithmetic-oriented hash function designed for efficient use inside proof systems [20].

Definition 2.1 (Valid private spend)**.** A public statement x is valid if there exists a private witness satisfying the range and value-conservation conditions, ownership and nullifier derivations, input and output note-commitment equations, and both Merkle-root equations described above.

2.3 Evaluated proof parameters

The prover commits to sixteen base-field columns and three extension-field columns. A nonzero challenge $\gamma \in \mathbb{K}$ combines these nineteen columns as $\sum_{i=0}^{18} \gamma^i c_i$. The initial codeword is evaluated on a circle domain over $\mathbb{F}_{2^{31}-1}$; four circle symbols form one query fibre. Four radix-four folds reduce it through three later line domains to a polynomial with four coefficients. Five salted Merkle trees authenticate the two initial commitments and the three later layers.

Table 2: Parameters of the evaluated proof.

Quantity	Value
Base field	$\mathbb{F}_{2^{31}-1}$
Challenge field	degree-four extension $\mathbb{K}$
Trace rows	2^{10}
Initial evaluation symbols	2^{19}
Initial query fibres	$2^{17} = 131{,}072$
Batched columns	19
Distinct queries	18, selected from at most 64 draws
Low-degree folds	four radix-four rounds
Final polynomial	four coefficients in $\mathbb{K}$
Proof-of-work difficulties	37, 34, 33, 30, 25, and 32 bits
Public-coin boundaries	30 after the initial commitment
Authenticated trees	five
Proof body	75,358 bytes

The circle-domain construction follows work on circle codes and Circle STARKs [14, 22, 23]. The evaluated profile uses its own four-way folding schedule, relation messages, bounded distinct-query sampler, and sequential work schedule; Section 6 gives the corresponding checked reduction.

2.4 Transcript and query selection

The Fiat–Shamir transform replaces the verifier's random challenges with hashes of the public transcript. The transcript absorbs the proof parameters and statement, commitment roots and public salts, algebraic claims, relation points and messages, later-layer roots, final polynomial, and six work nonces. Each nonce is tested against the current transcript state before being absorbed; the protected challenge is sampled only afterwards. Typed labels and length-delimited payloads separate the operations.

A work nonce succeeds when the corresponding SHA-256 digest has the required number of leading zero bits. Finding it takes repeated trials. The random-oracle model used in the security reduction treats the hash output on each new input as an independent uniform value while returning the same value on repeated inputs.

After every commitment, relation message, fold challenge, nonce, and final coefficient has been fixed, the verifier derives query material from SHA-256. It interprets each 32-byte block as eight little-endian 32-bit words, masks each word into the 2^{17}-element query domain, and keeps first occurrences. Verification proceeds only if eighteen distinct positions are obtained within sixty-four draws.

2.5 Atomic state transition

The proof body is sealed in a program-owned account before the spend transaction. The spend instruction validates the public statement and proof, checks that the nullifier has no existing marker, and then records both the marker and the new pool state in the same Solana transaction. Solana's transaction semantics make these writes atomic when the instruction succeeds. A later cleanup transaction may close the retained proof account and refund its balance; the nullifier and pool update remain in state.

3 Formal model and checked mathematics

The formal development is written in Lean 4 on top of mathlib [16, 32]. This section gives the mathematical core in the order used by the soundness proof: extract a spend witness from an algebraic trace, establish the concrete evaluation codes, carry one decoded candidate through four folds, connect those folds to the algebraic relation, authenticate every queried value, and prove the spend state transition. The implementation and probability arguments consume these theorems in Sections 5 and 6.

3.1 From algebraic rows to a spend witness

The proof system opens normalized columns encoding range checks, value balance, asset equality, ownership, note construction, nullifier construction, public fields, and two Merkle paths. The formal model records the residual equation for every constraint. Hash rows contain the intermediate states of the Poseidon2 permutation, while each Merkle level records its sibling and the left/right position selected by the path bit.

Theorem 3.1 (Trace soundness for the spend relation)**.** *Let x be a public statement and let O be the normalized columns opened by the proof. Suppose that all specified arithmetic residuals vanish, the opened hash rows follow the specified Poseidon2 round function, both depth-twenty paths are well formed, and the six spend fields in O equal the corresponding fields in x. If the deployed note, nullifier, and Merkle-node functions implement that round function on the reached inputs, then x has a witness satisfying Definition 2.1.*

The Lean proof constructs the witness from the opened columns. It converts the field balance equation into Equation (3) using the 2^{30} range bounds, rebuilds both twenty-level hash chains from their intermediate rows, derives ownership and the nullifier from the input-note opening, and obtains both note equations from the same trace. The range argument is essential: since each quantity is below 2^{30}, equality in $\mathbb{F}_{2^{31}-1}$ lifts to equality of the represented integers with no modular wraparound.

This theorem supplies the final semantic step of extraction:

$$\text{valid authenticated algebraic trace} \Longrightarrow \text{valid private-spend witness.}$$

The earlier steps establish that a successful verifier execution supplies that trace or enters one of the failure events counted in the security experiment.

3.2 Field, circle domain, and encoders

The base field is represented as $\mathbb{Z}/(2^{31}-1)\mathbb{Z}$. Lean constructs the unit circle as a commutative group and proves that the chosen generator has order 2^{31}. It follows that two powers of the generator have the same horizontal coordinate exactly when their exponents agree up to sign. The degree-four challenge field is built as a fixed tower over $\mathbb{F}_{2^{31}-1}$, including the packing basis and arithmetic identities used by the verifier.

The program stores circle evaluations in bit-reversed order and groups four symbols into each query fibre.

Theorem 3.2 (Concrete domains and encoders)**.** *The selected circle generator over $\mathbb{F}_{2^{31}-1}$ has order 2^{31}. Every point in the initial circle domain and three later line domains is distinct. Under bit-reversed storage, each query fibre maps to its four specified circle positions. All four encoders*

equal polynomial evaluation on their stated domains and in their deployed order. The later agreement bounds are $255, 63, 15, 3$*, which gives the required distance hypotheses.*

The order proof reduces equality of horizontal coordinates to equality of generator exponents up to sign. The encoder proofs then unfold the exact permutation and fibre maps used by the verifier. This establishes more than the cardinality of the domains: it identifies each stored symbol with the polynomial evaluation that the fold equations require.

The published circle-code decoding result is supplied at a named mathematical interface. Lean verifies its application to the concrete field, domain sizes, agreement threshold, list bound, nineteen batched columns, and maximum batching exponent eighteen. The general decoder theorem comes from the literature; every parameter used to instantiate it here is checked by the formal development.

3.3 One candidate through four folds

List decoding may return several nearby codewords. Soundness requires one initial candidate to remain coherent through all four verifier challenges; choosing a fresh candidate after each challenge would grant the prover an unpriced adaptive choice. The formal model therefore indexes each prover response by the transcript prefix available when that response is formed.

For each round $r \in \{1, 2, 3, 4\}$, Lean constructs a bad-challenge set B_r fixed by the preceding prefix and proves its cardinality bound. It then proves the following causal extraction result.

Theorem 3.3 (Coherent four-fold extraction)**.** *An accepted ideal low-degree execution either samples a challenge from one of the four prefix-determined sets* B_r*, or there is one member of the initial decoder list whose four exact radix-four folds equal the final polynomial published in the transcript.*

The extraction proof has complementary backward and forward parts. Starting from the four published coefficients, the backward argument selects close predecessors at dimensions 16, 64, 256, and 1024. Outside the four predecessor-failure events this gives one chain

$$c^{(0)} \in \mathbb{K}^{1024}, \quad c^{(r+1)} = F_{\alpha_r}(c^{(r)}), \quad c^{(4)} = c_{\text{final}}.$$

Only $c^{(0)}$ is chosen from the bounded decoder list; the later vectors are its folds, so the list-size factor is paid once. For Fiat–Shamir, the forward argument chooses a deterministic nearby candidate from each available transcript prefix. A transition from a state with no coherent continuation to one with a continuation can occur only when the next challenge lies in a bad set already fixed by that prefix. This is the causal statement needed by the oracle reduction.

The same four challenges update the weights used by the algebraic relation. Let $F_\alpha : \mathbb{K}^{4n} \to \mathbb{K}^n$ be the natural coefficient fold, let $D_\alpha : \mathbb{K}^{4n} \to \mathbb{K}^n$ be the verifier's dual weight fold, and let ∂ denote the boundary functional on the seven coefficients of a round polynomial.

Proposition 3.4 (Candidate and weight fold duality)**.** *For candidate values* $v \in \mathbb{K}^{4n}$ *and verifier weights* $w \in \mathbb{K}^{4n}$*, the deployed convolution constructs a polynomial* $P_{v,w}$ *of degree at most six satisfying*

$$\partial P_{v,w} = \sum_{i=0}^{4n-1} v_i w_i, \qquad P_{v,w}(\alpha) = \sum_{j=0}^{n-1} F_\alpha(v)_j D_\alpha(w)_j.$$

These identities hold at dimensions 1024*,* 256*,* 64*, and* 16 *with the coefficient and weight orders used by the Rust verifier.*

For one four-element fibre, the coefficient fold uses weights $(1, \alpha, \alpha^2, \alpha^3)$, while the dual fold uses $(1, \alpha^3, \alpha^2, \alpha)/4$. Expanding their product gives the seven coefficients of $P_{v,w}$. Summing this identity over fibres proves the proposition. The relation argument applies it in every round and bounds the challenge tuples capable of repairing a false scalar claim. Prover messages may depend on earlier challenges, while each repair set is fixed before the challenge charged to it.

3.4 Byte-level transcript

The transcript is an ordered sequence of typed operations. An absorb event contains a label and exact byte payload; a squeeze event records the requested field element, point, selector, or query block; a work event records its nonce and difficulty. These operations expand to primitive SHA-256 calls. An absorb advances the transcript state once, a squeeze uses separate output and state-advance hashes, and a work check examines its nonce without advancing the state.

Lean proves the complete operation order, event counts, labels, payload lengths, unique positions, and the check-before-absorb order for all six work nonces. It also proves that the challenges later consumed by the low-degree and relation checks are precisely the values produced at their transcript positions. The translated Rust path decodes the fixed proof-body slices into the same transcript inputs; Section 5 connects that decoder to the formal schedule.

3.5 Authenticated openings

Five Merkle commitments authenticate the queried values. An opened record is the value bytes followed by a 32-byte public salt. Leaf, binary-node, and radix-four-node hashes have separate domains, include the tree identity, and preserve child order. The two initial trees use query index q; the three later trees use $q \gg 2$, $q \gg 4$, and $q \gg 6$. Sorted, duplicate-free indices determine the compressed frontier. Each odd-depth path ends in a binary cap selected by the remaining index bit.

For each tree, the exact parser trace records every opened record, path slot, frontier node, and unconsumed suffix. The five-tree routine feeds each suffix into the following parser and requires the final suffix to be empty. From these traces, Lean constructs an authenticated forest and proves that every value consumed by the low-degree checker is the value portion of one of its authenticated records.

Theorem 3.5 (Authenticated consumption)**.** *Fix the five roots and the eighteen query positions. If the opening routine succeeds, every low-degree value read by that execution belongs to the exact record authenticated for its tree and position. Two successful forests that expose different reached values under those roots yield a concrete SHA-256 collision witness.*

This collision-witness formulation avoids an injectivity assumption for a compressing hash function.

3.6 State semantics

The state model assigns five accounts to fixed roles and checks their keys, owners, writable flags, current pool root and sequence, marker availability, and derived marker address. On success it writes the nullifier marker, updates the root, increments the sequence, and returns the exact resulting state. A pre-existing marker causes rejection, so two sequential modeled successes cannot use the same nullifier. Rejection leaves the modeled visible state unchanged.

A separate cleanup transition closes the proof account, transfers its full modeled balance to the supplied refund account, and preserves the pool and nullifier state. The deployed interpretation

Table 3: Main machine-checked mathematical results and their role.

Result	Checked content	Use in the security argument
Spend extraction	Arithmetic, ownership, notes, nullifier, public fields, and both Merkle paths imply a witness for Definition 2.1.	Turns a valid extracted trace into a valid private spend.
Domains and encoders	Circle-generator order, distinct evaluation points, stored ordering, encoder identities, and later-code distances.	Discharges the concrete hypotheses of the decoding argument.
Four-fold extraction	One coherent initial candidate reaches the final polynomial, or a prefix-timed bad challenge occurs.	Prevents adaptive switching between decoder candidates.
Fold duality	The degree-six round polynomial carries the current candidate dot product to the candidate and weight folds.	Joins low degree to the algebraic relation.
Transcript correctness	Exact byte payloads, operation order, work placement, and challenge consumption.	Supplies the causal random-coin schedule.
Opening correctness	Exact five-tree parsing and authentication of every value consumed by the low-degree checker.	Binds checked values to committed proof data.
State correctness	Exact successful marker and pool update, replay rejection, rollback in the state model, and cleanup preservation.	Gives a one-time atomic ledger transition.

of account locking, rollback, and finalized persistence is isolated in the runtime assumptions of Section 10.

4 Verification methodology

The proof was organized around the data actually received and produced by the verifier. This choice shaped both the Lean model and the translation work.

4.1 Prove local refinements, then one global execution

For each verifier stage we use three objects: the Rust computation, its functional translation, and a mathematical specification. Local refinement theorems connect the translated function to its specification. A final composition theorem starts from an arbitrary successful top-level call and threads the same execution record through every local theorem.

This last step is essential. Existential statements of the form "some accepted transcript has these challenges" and "some accepted opening has these values" can both be true while referring to different runs. The same-execution theorem instead proves

$$\rho \text{ succeeds} \Longrightarrow \bigwedge_k \mathcal{P}_k(\rho),$$

where each property $\mathcal{P}_k$ is projected from the same ρ. Intermediate equalities are obtained by reducing the translated code, not by adding them to the theorem's hypotheses.

4.2 Begin with proof bytes

The chain verifier receives roots, selected values, salts, and compressed authentication paths. It never receives complete committed tables. The formal opening model therefore begins with the literal byte stream. A successful section proof must account for every value, salt, path slot, and frontier node, and must return its exact unused suffix. The five-section routine passes those suffixes in order and accepts only an empty final suffix.

This makes omissions, duplicate records, wrong tree order, and trailing bytes ordinary failed proof obligations. Starting from a hypothetical full table would hide all four cases behind an unproved parser assumption.

4.3 Preserve causality

Every prover message is indexed by the transcript prefix available when the message is chosen. Each bad-challenge set is likewise fixed before its challenge is sampled. This discipline is used in the four-fold extraction, the relation repair bound, and the multi-attempt oracle argument.

The same rule applies at byte level. The transcript model records absorbs, squeezes, and work checks as distinct operations. The nonce is checked against the pre-absorb state, then absorbed, and only then may the next challenge be produced. Exact order is part of the theorem rather than a comment on the implementation.

4.4 Join authentication and algebra explicitly

Merkle authentication proves that a value belongs to a commitment. Low-degree testing proves an algebraic property of queried values. Neither fact implies the other. A consumed-value map connects them by proving that each value read by the fold and relation code is the value returned by the corresponding authenticated opening in the same run.

Public-statement binding receives the same treatment. The relation theorem uses nineteen columns in a fixed order, and the public-field rows identify six of those values with the spend statement. Lane batching, relation folding, and public binding are separate lemmas joined only at the final theorem.

4.5 Extract collision witnesses

SHA-256 and Poseidon2 compress their inputs, so their formal interfaces do not assume injectivity. When two reached inputs have the same output, the proof either establishes equality of the inputs or returns the unequal pair as a collision witness. The security experiment can then charge the collision event under a concrete primitive assumption.

Merkle paths add a positional condition. Comparing two different leaves at the same position identifies the first unequal ordered node inputs with one equal parent. Paths at different positions remain separate cases in the theft game.

4.6 Keep probability units visible

Every false-acceptance event lives in one probability space, and the detailed ledger has a proved union equality with its grouped form. This prevents a conditioning change from being hidden by prose. It also forces the analysis to distinguish work-normalized probability from the probability of failure after the work search has finished. The factor 2^{37} belongs to one of those experiments, not both.

4.7 Audit the proof as an artifact

The replay procedure resolves the dependency closure of the final theorem from a clean checkout. It rejects admitted proofs, unchecked native evaluation, and generated files absent from the manifest. It then compiles the closure and prints the final axiom report. Translation inputs, generated code, build tools, deployed bytes, proof bytes, and transaction records are checked by separate manifests.

This produces four kinds of evidence with different meanings:

Machine-checked theorem
: Lean accepts a statement under its displayed hypotheses.

Conditional reduction
: The conclusion follows when a published mathematical or cryptographic assumption is supplied.

Deterministic identity
: Recorded source, generated code, or program bytes match by replay and hash.

Execution record
: One archived transaction exhibits the stated input, cost, and state change.

The paper combines these forms of evidence without treating one as a substitute for another.

5 End-to-end implementation verification

The central engineering question is whether the mathematical statements in Section 3 describe the values accepted by the deployed verifier. We address this question for the successful proof-checking execution by combining functional translation of Rust with manually proved composition lemmas.

5.1 Deployed execution

The selected Rust path performs the following operations in order:

1. parse the fixed proof body and bind it to the live public statement;
2. replay the Fiat–Shamir transcript and six proof-of-work checks;
3. derive eighteen distinct query positions;
4. authenticate five sections of Merkle openings;
5. check four rounds of low-degree folding and the final polynomial;
6. decode and batch the algebraic claims, execute four relation rounds, and compare two final dot products; and
7. return success to the atomic state-transition code.

Verification must preserve both data identity and temporal order. For example, a correctly authenticated value from one run cannot justify an algebraic check in another, and a challenge cannot constrain a message that was absorbed after the challenge was sampled. We therefore model an execution record carrying all intermediate values and prove composition from one record.

5.2 Functional translation from Rust to Lean

Charon lowers selected safe Rust into a typed intermediate representation; Aeneas translates that representation into pure Lean functions [24, 25]. The translation preserves Rust control flow, pattern matching, arrays, slices, and mutable iterators by making state changes explicit in the generated function result. Generated functions are then related to the mathematical definitions by Lean theorems.

The complete Solana entry point also contains framework objects, system calls, account operations, and function pointers outside the translators' supported subset. The selected translation boundary therefore encloses the proof-checking computation, while the surrounding account transition is specified separately. This boundary is exact: the artifact guide records the selected Rust source, translator revision, generated Lean, and the theorem that reconnects the translated result to the calling verifier.

Translation found one proof-artifact defect. An early handwritten model of a mutable iterator rebuilt an array from the exhausted iterator and discarded the writes. Lean contains a counterexample to that model. The corrected model rebuilds the array from the updated original storage, and an extended translation of the unchanged Rust function produces the same behavior. The defect affected the proof model rather than the deployed program; retaining the counterexample makes this distinction auditable.

5.3 Accepted-execution evidence

For a translated execution ρ, let $\mathsf{Accept}(\rho)$ mean that the proof-checking call returns success. Let $\mathsf{Evidence}(\rho)$ collect the following facts about that same run:

- the parsed body and live statement determine the absorbed statement digest;
- every transcript challenge equals the output of its prescribed hash operation, and all six work conditions hold in order;
- query derivation returns eighteen distinct positions within its draw bound;
- all five opening sections authenticate, consume the exact byte stream, and supply the values later read by the verifier;
- four radix-four folds, their coordinates, and the final four-coefficient polynomial satisfy the low-degree checks;
- the decoder supplies exactly seventy-six point claims, four prepared claims, the initial relation value, and the fifty-eight-field message consumed by the four relation rounds; and
- the twelve-component main accumulator and four-component compact accumulator evolve through those same rounds and equal the two final dot products checked by the verifier.

Theorem 5.1 (End-to-end accepted-execution theorem)**.** *For every translated deployed execution ρ,*

$$\mathsf{Accept}(\rho) \Longrightarrow \mathsf{Evidence}(\rho).$$

All components of $\mathsf{Evidence}(\rho)$ *are derived from ρ. In particular, neither accumulator equality nor agreement with a separately executed mathematical verifier is a premise.*

The proof is compositional but not merely a conjunction of unrelated helper lemmas. Each stage receives the exact values returned by the preceding stage of ρ, and the final theorem uses one shared four-round execution for the authenticated values, fold challenges, relation messages, and both accumulators. This is the same-execution property introduced in Section 1.

The final helper may be represented by an arbitrary function. The theorem still obtains the downstream security evidence because the independently derived openings, folds, claims, and dot products contain everything later reasoning uses. Consequently, the security classification does not trust the helper's return value as a summary of unverified work.

5.4 Transcript and opening correspondence

For the transcript, the translated code is proved equal to the byte-level specification on every reached operation. The proof covers exact labels, payloads, endianness, field and point sampling, nonzero rejection, state advancement, and the placement of all work checks. It then connects each returned challenge to the value consumed by the fold or relation code.

For authenticated openings, the translated parser is inverted from its successful result. This yields the value–salt split, shifted query index, radix-four slots, binary cap, compressed frontier, reconstructed root, and literal remaining suffix for each section. The outer proof fixes the order of all five sections and proves that no bytes remain. A consumer theorem then identifies every low-degree read with a returned authenticated value in the same execution.

These are deterministic correspondence results. SHA-256's collision resistance and its random-oracle treatment enter later as cryptographic assumptions; the correspondence result establishes which concrete byte strings and hash calls those assumptions cover.

5.5 Algebraic correspondence

The algebraic proof begins with the exact decoded table rather than an abstract promise that a relation check succeeded. It proves the construction of four prepared claims, the initial combined value, the complete relation message, and the four ordered round calls. The low-degree and relation sides share the same four challenges and final coefficients.

The main accumulator has twelve terminal components, including dense and deferred grouped loops. The compact accumulator has four components and passes through a constructor, four fold cases, and final assembly. Lean proves both state evolutions and both final dot products. Their composition closes the last assumed algebraic equalities that previously separated local implementation lemmas from the final accepted-execution theorem.

5.6 Replay and auditability

The accepted-execution result has a tracked dependency closure of 331 Lean modules. The replay script resolves every dependency from a clean checkout, uses the pinned translator revision, rejects unfinished proof markers and unchecked native evaluation, builds the closure, and prints the axioms of the final theorem. The recorded replay uses Lean 4.32 and reports only Lean's standard logical foundations; it contains no admitted project theorem.

The source-to-binary record is a separate evidence layer. A clean build from the archived source and pinned Solana and Rust tools reproduces the deployed program byte for byte. The release archive then binds those bytes to the program identifier and finalized transaction. Full declaration names, file paths, commands, source revisions, and digests live in the artifact guide rather than in the manuscript.

Table 4: Coverage of the end-to-end theorem.

Stage	Evidence derived from any successful call
Parsing and statement	Fixed body layout, decoded fields, live-statement digest, and full input consumption
Transcript and work	Exact hash framing and challenges; six ordered work conditions
Queries and openings	Eighteen distinct positions; five authenticated sections and all values consumed later
Low-degree test	Four fold comparisons, coordinates, and final polynomial
Algebraic relation	Seventy-six decoded claims, four prepared claims, fifty-eight relation fields, and four rounds
Final checks	Twelve-component main state, four-component compact state, and both checked dot products

6 Security experiment and soundness

6.1 Threat model

The adversary chooses a public statement and proof after observing the public history. It succeeds when the verifier accepts but the statement has no witness satisfying Definition 2.1. The adversary may adapt its oracle queries and later messages to the transcript prefix already available. Its resources include SHA-256 oracle queries and the work spent searching for the six nonces.

All failure events are defined in one finite probability space (Ω, μ). Let $A_{\text{false}} \subseteq \Omega$ be the event of accepting a false statement. Let C be the protocol failure event, which combines the low-degree, query, relation, and work terms. The remaining events describe either a deterministic mismatch between the deployed execution and the formal experiment or a cryptographic failure such as a hash collision.

This single-experiment formulation permits one pointwise coverage theorem.

Theorem 6.1 (Finite failure decomposition)**.** *For every outcome in the security experiment, acceptance of a false statement implies either the concrete protocol failure event C or one of the eighteen external events:*

$$A_{\text{false}} \subseteq C \cup \bigcup_{i=1}^{18} E_i. \tag{4}$$

The detailed ledger contains each event exactly once, and its union is equal to the right-hand side of Equation (4)*.*

The detailed protocol ledger has six entries: the query and final-work event, four fold events, and relation repair. Lean proves that their union equals C. The eighteen external events are grouped in Table 5.

For the selected translated execution, Theorem 5.1 supplies transcript, relation, and opening correspondence. The other deterministic interfaces are recorded so that proofs can make their events empty instead of assigning them arbitrary small probabilities.

6.2 Symbolic decomposition

Taking measures in Equation (4) and applying the union bound gives

$$\Pr[A_{\text{false}}] \leq \Pr[C] + \sum_{i=1}^{18} \Pr[E_i]. \tag{5}$$

Table 5: External interface of the false-acceptance experiment.

Group	Count	Events
Implementation correspondence	5	Transcript translation, reached SHA-256 semantics, reached Poseidon2 semantics, relation implementation, and authenticated-opening implementation
Cryptographic primitives	5	SHA-256 collision, target preimage, and random-oracle replacement; Poseidon2 collision and target preimage
Credentials	1	Recovery of the victim's secret credential
State and runtime	7	Program/state correspondence, address and system program behavior, writable-account locking, rejection rollback, marker persistence, finalized observation, and cleanup/refund behavior

Lean proves this inequality over the exact ordered event list and proves that each event appears once. No independence assumption is used.

For the conventional cryptographic presentation, let D_{prod} assert all deterministic implementation, compilation, and runtime correspondences. Under D_{prod}, those mismatch events are empty. The remaining six probabilistic events are the five primitive events and credential recovery; writing them as $P_1, \ldots, P_6$ yields

$$\Pr[A_{\text{false}}] \le \Pr[C] + \sum_{j=1}^{6} \Pr[P_j] \qquad \text{under } D_{\text{prod}}. \tag{6}$$

This split preserves the larger audit ledger while giving deterministic claims their natural logical role.

6.3 Bounded distinct-query sampling

The query sampler is modeled exactly, including its sixty-four-draw limit and first-occurrence rule. Let $S(W)$ retain the first eighteen distinct elements of a word sequence $W \in [N]^{64}$, returning $\perp$ when fewer than eighteen distinct values appear.

Theorem 6.2 (Exact capped distinct-query law)**.** *If the words of W are uniform in $[N]$, then conditioned on $S(W) \neq \perp$, every ordered injection $[18] \hookrightarrow [N]$ has the same probability. For every fixed $B \subseteq [N]$ of size $18 \le A \le N$,*

$$\Pr[S(W) \neq \perp \ \wedge \ \operatorname{im} S(W) \subseteq B] \le \frac{(A)_{18}}{(N)_{18}} = \prod_{j=0}^{17} \frac{A-j}{N-j}. \tag{7}$$

Here $(m)_k = m(m-1)\cdots(m-k+1)$. For any two ordered injections, a permutation of $[N]$ maps the complete preimage of one under S bijectively to the preimage of the other. This proves conditional uniformity despite the variable number of duplicate draws. There are $(A)_{18}$ injections into B and $(N)_{18}$ injections in total. Sampler exhaustion rejects, which gives the unconditioned inequality in Equation (7).

For the verifier, $N = 131{,}072$ and $A \le 6{,}082$. Lean checks the falling-factorial identity and the concrete inequality

$$\frac{(6082)_{18}}{(131072)_{18}}\, 2^{-32} \le 2^{-111}, \tag{8}$$

where the displayed 2^{-32} factor belongs to the separately modeled final work condition.

6.4 Low-degree and relation reductions

The low-degree proof has three layers. First, the formalization checks that the concrete encoders meet the field, ordering, degree, distance, agreement, and multiplicity hypotheses of the cited circle-code decoding result. The general decoding theorem remains a literature assumption.

Second, Theorem 3.3 preserves one decoder candidate through all four folds. The backward proof constructs the exact coefficient chain; the forward proof assigns each exceptional challenge set to the transcript prefix that fixes it. A separate adaptive-oracle theorem shows that Q completed fresh attempts cost at most Q times the one-attempt bound, even when the next attempt depends on the full previous history.

Third, the algebraic relation connects the extracted candidate to Definition 2.1. Nineteen columns are batched with a nonzero extension-field challenge of maximum exponent eighteen. For a supplied family of L candidate strategies, the checked relation argument places all repairing twelve-challenge tuples in a set of relative size at most

$$\frac{32L}{|\mathbb{K}|}, \qquad L \le 240. \tag{9}$$

The sequential construction permits each later message to depend on the earlier challenges already revealed. Theorem 3.4 proves that the candidate coefficients and verifier weights carry the same scalar claim through each fold. The final semantic step is Theorem 3.1, which constructs a private-spend witness from the resulting trace.

The Fiat–Shamir step uses the published state-restoration analysis for interactive oracle proofs [7, 12, 14]. The byte-level transcript proof determines the exact oracle inputs and causal schedule. The random-oracle replacement and applicability of the published theorem are explicit assumptions at this boundary. Appendix D records the exact published result, equation, concrete parameter substitution, and Aspis predicate at each such interface.

6.5 Work-normalized accounting

The six proof-of-work checks restrict the fraction of transcript attempts that reach their protected challenges. In a work-normalized experiment, repeated trials are charged to the adversary. With nineteen columns, maximum batching exponent eighteen, thirty public-coin boundaries, and the work difficulties in Table 2, Lean verifies the concrete rational arithmetic and proves

$$B_{\text{protocol}} \le \frac{7}{10 \cdot 2^{100}}. \tag{10}$$

This inequality is conditional on the checked decoding applicability, code-membership connection, authenticated openings, Fiat–Shamir theorem, and grinding-output model described above.

Let B_{external} bound the six probabilistic events remaining under D_{prod}, and assume

$$B_{\text{external}} \le \frac{3}{10 \cdot 2^{100}}. \tag{11}$$

Theorem 6.3 (Conditional work-normalized soundness)**.** *Assume the event coverage of Equation* (4)*; the checked connection from the exact protocol event to the concrete arithmetic; the published decoding and Fiat–Shamir results with their verified parameter hypotheses; the deterministic correspondences* D_{prod}*; and primitive and credential bounds satisfying Equation* (11)*. Then*

$$\Pr[A_{\text{false}}] \le 2^{-100}.$$

The final step is ordinary arithmetic:

$$\Pr[A_{\text{false}}] \le B_{\text{protocol}} + B_{\text{external}} \le 0.7 \cdot 2^{-100} + 0.3 \cdot 2^{-100} = 2^{-100}.$$

The substantive checked content is the preceding pointwise classification, event-ledger equality, parameter application, and protocol arithmetic.

6.6 Raw resource-bounded accounting

Work-normalized probability and probability after a completed grind answer different questions. Let $1 \le Q \le 2^{128}$ be the adversary's SHA-256 oracle-query budget, let $R \le 32$ bound the public-coin boundaries, and let ε_{I} bound each interactive protocol branch. The state-restoration specialization represented in Lean is

$$b(Q) = \left(1 + \frac{R}{Q}\right)\varepsilon_{\text{I}} + 3\left(Q + \frac{1}{Q}\right)2^{-256}.$$

Its corresponding resource-bounded form is

$$\begin{aligned}\mathrm{Adv}^{\text{false}}_{\text{ASPIS}}(\mathcal{A}; Q) \le\ & 3(Q+R)\varepsilon_{\text{I}} + 9(Q^2+1)2^{-256} \\ & + \varepsilon_{\text{prim}}(\mathcal{A}, Q) + \varepsilon_{\text{cred}}(\mathcal{A}),\end{aligned} \tag{12}$$

under D_{prod} and the same decoding and Fiat–Shamir assumptions. The two final functions collect the primitive and credential advantages and are left symbolic.

Once the 37-bit initial work search has been completed, its factor cannot also be charged in the exponent. Removing that normalization leaves the dominant explicit term at roughly 70–71 bits, plus the displayed primitive, credential, and runtime terms. We report this raw view alongside Theorem 6.3 so the unit of every security figure remains clear.

7 Theft resistance and state

Soundness concerns false statements. Theft resistance adds a target: a live note owned by someone else. It also depends on what the attacker observes, how hash collisions are classified, and whether a successful nullifier remains spent on chain. We formalize these questions in two games and an explicit state machine.

7.1 Fixed-victim game

The first game fixes a live note, its tree position, and its depth-twenty path before the attack. The target includes a valid private opening. Its public commitment, root, and nullifier are computed with the same functions used by the spend relation.

An accepted proof targets this note if it either publishes the victim's nullifier or opens a leaf at the victim's exact position under the victim's root. The following case split is proved in Lean.

Theorem 7.1 (Fixed-victim classification)**.** *Every accepted attack on the fixed victim yields at least one of the following:*

1. *extraction fails to produce a valid spend witness;*
2. *the extracted witness contains the victim's secret credential and nullifier randomness;*
3. *a different secret opening produces the victim's nullifier;*

4. a different valid note opening produces the victim's commitment; or

5. a different leaf at the same tree position produces the victim's root.

In the fifth case, the two paths expose distinct ordered inputs to one Merkle-node hash with the same output.

The same-position condition is necessary. Two direction-bit sequences can name two legitimate leaves of one tree, so equality of their roots alone is not a collision witness for a single node call. Fixing the position lets the proof compare the paths from the leaves upward and identify the first unequal node inputs with an equal parent.

7.2 Adaptive public history

The second game gives the attacker an arbitrary finite ordered history of public proof records. After seeing its public projection, the attacker chooses a spend attempt as a function of the whole observed prefix. Success is the first fraudulent spend selected by this experiment.

Lean lifts Theorem 7.1 to this setting with a history-dependent extractor. It retains a separate event for an invalid or ambiguous victim setup, then maps the remaining cases into the false-acceptance, credential, hash, state, and runtime events of Section 6. The result applies to finite histories; a system that admits several final attack attempts must include their number in its experiment or use a suitable state-restoration bound.

For an accepted submitted proof, the work search has already happened. The raw accounting therefore omits the 37-bit normalization. Its refined form is

$$\begin{aligned}\Pr[\text{first fraudulent spend}] \le 2^{-70} + B_{\text{statement}} + \Pr[\text{hash or Merkle residual}] \\ + B_{\text{primitive}} + B_{\text{credential}} + B_{\text{runtime}} + \Pr[\text{invalid setup}].\end{aligned} \tag{13}$$

Lean checks the reduction and the explicit 2^{-70} arithmetic. The other terms remain visible so that the formula cannot be mistaken for a closed numerical theft bound.

7.3 Nullifier-controlled transition

The modeled spend transition performs five groups of checks:

1. the five supplied accounts have the required keys, owners, and writable flags;
2. the pool root and sequence match the public statement;
3. the nullifier maps to the supplied marker address;
4. the marker account is fresh or prepared but empty; and
5. the proof checker has accepted the same public statement.

On success, the transition writes the nullifier into the marker, replaces the pool root, and increments the sequence. Lean proves the exact resulting state, rejection of an occupied marker, and the impossibility of two sequential modeled successes with the same nullifier. Even if two different nullifiers map to the same marker address, the first occupied account prevents the second transition from overwriting it.

The marker prevents replay after a committed spend. The extraction and theft games handle the separate question of whether the first spend was authorized.

7.4 Cleanup, rollback, and finality

The proof account may be closed in a later transaction. The cleanup model checks the proof and refund accounts, transfers the full retained balance to the supplied refund account, invalidates the proof account, and leaves the pool and marker unchanged. A trace theorem places the spend writes before this later refund.

Rollback is exact in the abstract state machine: a rejected instruction has the original visible state. Applying this result to Solana uses the runtime premises for writable-account locking, system-program behavior, transaction rollback, committed marker persistence, and finalized observation. The recorded transaction in Section 9 supplies a concrete execution of the success and cleanup paths; the universal claims remain at the formal interfaces listed in Section 10.

8 Hiding model

The public spend view contains the old root, nullifier, output commitment, new root, asset, fee, deployment domain, and proof data. The intended private data are the input-note opening and credential, tree path, output opening, and internal algebraic trace. The formal development proves witness independence for the finite algebraic masking model and records the comparisons needed to carry that statement to the deployed byte encoding.

8.1 Algebraic witness independence

The masking construction has four components: the main trace, copied consistency columns, boundary columns, and an auxiliary component that satisfies the final linear condition. Uniform masks are sampled from finite spaces. Surjective linear maps show that the visible masked values have the same distribution for any two compatible witnesses.

The concrete sampler operates on whole 32-bit words. It uses first success under rejection sampling, with at most sixteen words for each field limb. The formal view includes this sampler, the degree-four extension-field packing, the kernel correction, the downstream encoding, and an arbitrary deterministic serialization of the resulting view.

Theorem 8.1 (Fixed-schedule witness independence)**.** *Fix a public statement, transcript schedule, and two private witnesses that satisfy the same statement. Under the finite sampling and linear-algebra hypotheses of the masking construction, the encoded algebraic views generated from the two witnesses have identical distributions.*

The equality is exact in the finite model. Any deterministic encoding preserves it.

8.2 Bounded retry

The prover may test up to seventeen masking schedules and publish the least-indexed successful one. Selection is part of the public distribution, so the model includes the retry controller. Lean proves that the released selector and chosen view remain witness-independent when each releasable successful branch has the same law for both witnesses.

The implementation interface for this theorem identifies the real entropy source, word rejection sampler, proof-or-abort event, retry cap, selection rule, and public release with their modeled counterparts. Stating retry at this level also exposes timing or abort behavior that a fixed-schedule statement would miss.

8.3 Byte inventory and hybrid statement

The modeled public encoding consists of a 169-byte instruction, a 40-byte sealed-proof header, and a 75,358-byte proof body, for 75,567 bytes in total. Lean proves that every offset belongs to one declared field or semantic class and that splitting and concatenation are inverse operations. This establishes a complete byte inventory for the modeled encoding.

To compare deployed views, the formalization uses a two-sided hybrid. Each side passes through the Fiat–Shamir compiler, Rust transcript, SHA-256 calls, serialization, salted Merkle commitments, Poseidon2 calls, entropy expansion, rejection sampling, retry, and the final algebraic view. If the error assigned to these steps on side b is B_b, then

$$\Delta(V_0, V_1) \leq B_0 + B_1, \tag{14}$$

where Δ is statistical distance and Theorem 8.1 supplies equality at the center of the hybrid.

Equation (14) is a conditional statement about the deployed view. A computational hash assumption naturally bounds an adversary's distinguishing advantage, while statistical distance compares full distributions. A concrete implementation theorem must use one metric consistently and prove the Rust, serialization, entropy, retry, and commitment comparisons in that metric. The current checked result is exact witness independence for the finite masking model together with the explicit hybrid interface above.

9 Evaluation and reproducibility

We evaluate the work at three levels: replay of the formal theorem, identity of the deployed program, and one finalized Solana execution. The associated artifacts are published with the source repository [4].

9.1 Formal replay

The final accepted-execution theorem has a dependency closure of 331 tracked Lean modules. The release replay starts from a clean checkout, verifies that every required source is tracked, applies the pinned Rust translator, rejects unfinished proof markers and unchecked native evaluation, and builds the entire closure with Lean 4.32. It then prints the theorem's axioms. The recorded result contains Lean's standard logical foundations and no admitted project theorem.

The replay is intentionally narrower than building every historical file in the repository. Its manifest is generated from the final theorem's actual imports, so passing the replay establishes that the published dependency closure is complete and self-contained.

9.2 Program identity

The deployment archive contains the clean source tree, lock file, build command, Solana platform tools, Rust toolchain, program binary, proof body, public statement, and signed transaction bytes. A clean build with the recorded tools reproduced the 1,258,496-byte program exactly. SHA-256 digests for every artifact and the complete source revision are listed in the artifact guide.

This record gives a checkable chain

$$\text{archived source} \longrightarrow \text{program bytes} \longrightarrow \text{program identifier} \longrightarrow \text{transaction}.$$

The first arrow is checked by a deterministic rebuild and byte comparison. The second and third are checked against the captured loader and transaction records.

Table 6: Recorded evaluation results.

Measurement	Result
Formal dependency closure	331 tracked Lean modules
Replay environment	Lean 4.32 with pinned Rust-to-Lean translator
Program binary	1,258,496 bytes; clean build reproduced byte for byte
Proof body	75,358 bytes
Statement artifact (JSON)	768 bytes
Finalized spend	slot 435,019,536
Compute usage	1,334,452 of 1,356,912 units
State effect	nullifier marker created; root and sequence updated atomically
Cleanup	525,660,961 lamports refunded; spend state preserved

9.3 Mainnet execution

This execution is the chain result underlying the priority claim in Section 1. The program itself performed the transparent proof verification and, on success, wrote the nullifier marker and new pool state in the same transaction.

The archived mainnet transaction finalized at slot 435,019,536. It read the sealed proof account, verified the 75,358-byte proof, and recorded the nullifier and new pool state in 1,334,452 compute units. The transaction limit was 1,356,912, leaving 22,460 units of headroom. Exact signed-wire simulation reported the same cost.

The before-and-after account snapshots record the old root and sequence, the new root and sequence, and creation of the nullifier marker. A later cleanup transaction closed the retained proof account and refunded 525,660,961 lamports while preserving the spend state. The deployment was subsequently closed, so the release archive includes the RPC responses and their digests needed to inspect the historical accounts.

The demonstration pool contained one note. This execution therefore measures proof verification, compute cost, and the atomic state transition. Privacy is addressed by the distributional model of Section 8, not by the size of this demonstration's anonymity set.

9.4 How the evidence supports the claim

Formal replay checks a universal theorem about the translated successful execution. Build reproduction identifies the program bytes to which the source record refers. The transaction supplies one concrete execution with a measured cost and observed state change. The conditional security theorem then states the mathematical and cryptographic premises under which an accepted false statement is bounded. Each layer answers a different audit question, and the artifact guide provides the exact command or digest for checking it.

10 Assumptions and claim boundary

The end-to-end theorem begins with a successful execution of the translated deployed proof checker and ends with the complete evidence consumed by the formal security classification. The surrounding upload, account, cleanup, compiler, and chain layers have their own specifications and artifact records. Two open composition statements sit directly on the path to a deployed probability theorem:

1. **Statement identification.** The translated entry path checks the live statement against its digest. A field-by-field theorem must identify that object with the abstract statement used by

Table 7: Assumptions used by the paper's claims.

Interface	Role	Evidence supplied here
Circle-code decoding	Turns proximity at the evaluated parameters into a bounded candidate list	Lean checks every concrete parameter and encoder hypothesis against the published proposition
Fiat–Shamir analysis	Converts the causal interactive argument to the SHA-256 transcript with a query budget	Exact byte schedule and public-coin count are checked; the published state-restoration theorem is assumed
SHA-256	Transcript oracle and Merkle binding	Exact reached inputs and Solana hash-call framing are specified; collision, preimage, and random-oracle bounds are cryptographic assumptions
Poseidon2	Notes, nullifiers, and trace Merkle nodes	Round function and reached-input correspondence are specified; concrete collision and preimage bounds are assumptions
Credentials	Authorizes the input note	Theft reductions isolate recovery of the victim secret as its own event
Translation and proof tools	Connect selected Rust to Lean and check the theorems	Pinned translation, complete dependency replay, and axiom report
Compiler and build	Connect recorded source to Solana program bytes	Clean reproducible build and byte comparison
Solana runtime	Account locking, address creation, rollback, persistence, and finalized observation	Explicit state model plus archived transaction and account records
Deployed-view privacy	Connects the finite masking law to deployed views	Exact modeled hiding theorem and a two-sided hybrid with named comparison errors

the false-acceptance and theft experiments.

2. **Experiment lift.** The deterministic classification of one successful call must be embedded in the causal probability law used for work-normalized accounting, including oracle queries, rejection sampling, and repeated attempts.

The accepted-execution theorem derives the mathematical consequences of verifier success. Separate completeness theorems cover the honest selector and work schedule, while the archived proof supplies a complete successful execution of the deployed program.

All other assumptions enter through the interfaces summarized below.

10.1 Poseidon2 parameter status

The construction uses Poseidon2 over $\mathbb{F}_{2^{31}-1}$ with width sixteen, exponent five, eight full rounds, and fourteen partial rounds [20]. Recent algebraic analysis improves attacks on several Poseidon2 variants and recommends evaluating each parameter set separately [27]. That work reports no full-round break for the recommended instances it studies and places its ideal-degree estimates above 2^{128} when the exponent is at least three. It does not publish a concrete advantage for the exact Aspis tuple. We therefore keep the Poseidon2 collision and preimage terms symbolic.

10.2 Interpretation

The formal result is end to end for the selected successful proof-checking execution. The main numerical statement is the conditional work-normalized theorem of Theorem 6.3; the raw statement is Equation (12). The mainnet transaction in Section 9 identifies the binary, proof, cost, and state transition used for evaluation. These three statements have distinct scopes and share the artifact links needed to audit their connections.

11 Related work

11.1 Transparent arguments

Aspis builds on interactive oracle proofs and the FRI family of Reed–Solomon proximity tests [7–9]. DEEP-FRI improves proximity testing with out-of-domain samples, while WHIR combines proximity testing with multilinear evaluation claims [3, 10]. Circle codes and Circle STARKs provide smooth evaluation domains over Mersenne fields [22, 23]. The S-two whitepaper gives the closest published concrete circle-code protocol and Fiat–Shamir accounting [14].

The evaluated Aspis verifier has a distinct four-way fold, two initial commitments, three later Merkle layers, bounded query sampler, algebraic relation, and six-step work schedule. The Lean development proves its encoder identities, causal candidate chain, query distribution, and relation composition for those parameters. It imports the stated circle-decoding and Fiat–Shamir results at explicit interfaces. Work on proximity gaps and mutual correlated agreement supplies the mathematical setting for the candidate analysis [11, 13, 15, 21].

11.2 Machine-checked cryptography and implementations

EasyCrypt established a practical framework for machine-checked game-based cryptographic proofs [5]. Machine-checked MPC-in-the-head work proves security for a general zero-knowledge construction and connects it to executable components [2]. Jasmin, HACL*, and Fiat-Crypto demonstrate strong links between proof and efficient cryptographic code through verified compilation, extraction, or generation [1, 18, 36].

For proof systems, ArkLib develops a modular Lean theory of interactive oracle reductions, composition, and knowledge soundness [33]. Work on verifying deployed zero-knowledge verifiers stresses the need to connect an abstract security proof to the code that parses and accepts proofs [19]. Aspis carries this connection through a complete deployed acceptance path. Its universal theorem starts from the successful result of translated Rust and derives the transcript, authenticated openings, four low-degree rounds, relation execution, and both final dot products from that same call. The theorem is published together with a reproducible binary and a finalized transaction. This source-to-security span, rather than any single refinement lemma, is the formal novelty of the work.

11.3 Private payments and Solana

Zerocash established the note-commitment and nullifier pattern used by modern shielded payments [6]. Solana's Token-2022 Confidential Transfer extension uses client-generated equality, ciphertext-validity, and range proofs verified by the native ZK ElGamal proof program, followed by an encrypted-balance update; the proof program is enabled on mainnet and devnet [31]. Elusiv previously released a mainnet private-send SDK [17].

Among hash-based proof systems on Solana, Protocol 01 publishes a transparent STARK-based shielded-payment stack and documented devnet verifier [30]. The solana-pqzk-fullchain

Table 8: Closest public work along the axes combined here.

Work	Transparent	Private spend	Chain state	Machine-checked security	Executable-code link
Zerocash [6]	no	yes	protocol design	no	reference implementation
Machine-checked MPC-in-the-head [2]	varies	no	no	yes	extracted and verified components
ArkLib [33]	yes	no	no	framework	implementation work in progress
Token-2022 confidential transfer [31]	yes	encrypted balances	mainnet proof program	no	native programs
Protocol 01 [30]	yes	yes	separate devnet steps	no	public Rust
solana-pqzk [35]	yes	no	devnet verification	no	public Rust
HFIPay [34]	yes	claim authorization	aggregation design	no	reference backend
Privacy Cash [28, 29]	no	yes	mainnet	no	public program
Aspis	yes	yes	atomic mainnet record	conditional Lean development	translated successful path

project directly verifies a Winterfell STARK on devnet and reports reproducible costs for an affine-counter statement [35]. HFIPay describes a transparent Circle STARK backend for private claim authorization; its direct verification path uses Groth16, while the transparent backend is assigned to aggregation [34]. Privacy Cash records a mainnet shielded-state deployment using Groth16, and Light Protocol publishes setup material for its Groth16 proving stack [26, 28, 29].

To our knowledge, following the public-evidence search completed on 24 August 2026, Aspis is the first publicly evidenced Solana mainnet transaction in which a program directly verifies a transparent, computationally hiding private-spend proof and atomically records the corresponding nullifier and pool-state update within the transaction compute limit. It is also the first such Solana result we found with an end-to-end machine-checked theorem for the deployed successful proof-checking path and reproducible source, binary, proof, and transaction evidence. The comparison is dated and covers publicly indexed papers, repositories, deployments, and patents.

12 Conclusion

Aspis establishes a transparent private spend on Solana mainnet and gives it a machine-checked account from deployed code to cryptographic evidence. To our knowledge at the stated search cutoff, its finalized transaction is the first publicly evidenced Solana mainnet result to directly verify a transparent, computationally hiding private-spend proof and atomically record the corresponding nullifier and new pool state. The 75,358-byte proof was verified in 1,334,452 compute units.

The formal result is unusual in its reach. Given any successful translated call to the deployed Rust proof checker, Lean derives the statement digest, byte transcript, six work checks, eighteen queries, five authenticated opening sections, four low-degree folds, final polynomial, complete relation data, and both final accumulator equalities. One execution supplies every value. The theorem therefore covers the full acceptance computation rather than a collection of components whose composition is left to inspection.

The mathematics beneath that theorem is concrete and complete for the evaluated verifier. It constructs a private-spend witness from the trace, proves the Mersenne-field domains and encoders, derives the exact capped-query law, extracts one coherent polynomial candidate across four folds, proves the candidate and weight duality used by the relation, and connects acceptance to theft,

replay, and state-transition results. A single finite experiment then classifies false acceptance. Published decoding and Fiat–Shamir results give a work-normalized protocol budget of at most $0.7 \cdot 2^{-100}$; the stated external budget and correspondences yield the conditional 2^{-100} total, with raw post-grinding accounting reported separately.

The theorem replay, pinned translation, byte-identical program build, proof and statement bytes, transaction, and account snapshots make each boundary independently checkable. The remaining fieldwise statement identification and causal probability lift are explicit composition interfaces. Taken together, the result shows both that transparent private spending fits within Solana mainnet's execution limit and that the complete path from proof bytes to its cryptographic evidence can be verified at deployed-code granularity.

A End-to-end proof outline

This appendix gives the logical shape of Theorem 5.1. Exact Lean declaration names and source locations are listed in the external artifact guide.

Let ρ be one execution of the translated proof checker. The success equation is inverted stage by stage.

1. **Parsing.** Success fixes the body length, every decoded field, the statement digest, and the absence of trailing fixed-body bytes.

2. **Transcript.** Functional reduction identifies each absorb, squeeze, state advance, nonce test, and derived challenge with the mathematical transcript operation at the same position.

3. **Queries.** The generated sampler proof yields eighteen distinct positions and the exact first-occurrence order used by ρ.

4. **Openings.** Inverting the five successful parser calls yields exact section traces. Their suffix equations compose, and the last suffix is empty. Root equations produce the authenticated forest.

5. **Consumption.** Array-index and lookup lemmas identify each value read by the low-degree checker with the corresponding value in that forest.

6. **Folding.** The four unrolled verifier rounds determine their coordinates, opened quartets, challenges, and folded values. The last comparison fixes the four final coefficients.

7. **Relation.** Decoding fixes the seventy-six claim values and fifty-eight relation fields. The four prepared claims and initial relation value are computed from those same values. Four source rounds use the fold challenges already obtained from ρ.

8. **Accumulators.** Loop and iterator invariants derive all twelve main components and four compact components. Their terminal dot products are exactly the two comparisons made by the verifier.

9. **Composition.** Substitution joins the preceding equalities in one execution record and produces $\mathsf{Evidence}(\rho)$.

The final theorem quantifies over the last helper implementation because its return value is absent from the derived evidence. The proof instead uses the opening, fold, relation, and accumulator equations established before that call.

B Coherent extraction and relation proof

This appendix gives the central algebraic argument behind Theorems 3.3 and 3.4. Let Y_r be the received word at layer r, let E_r be its checked encoder, and let F_{α_r} be the coefficient fold at round r. The final layer is the published vector $c^{(4)} \in \mathbb{K}^4$.

For the backward argument, suppose the query miss and four predecessor events do not occur. Since the eighteen accepted queries are consistent and the global consistency set exceeds the decoded agreement threshold, the final vector has a close predecessor $c^{(3)} \in \mathbb{K}^{16}$ satisfying $F_{\alpha_3}(c^{(3)}) = c^{(4)}$. Repeating the predecessor theorem gives

$$c^{(2)} \in \mathbb{K}^{64}, \qquad c^{(1)} \in \mathbb{K}^{256}, \qquad c^{(0)} \in \mathbb{K}^{1024},$$

with $F_{\alpha_r}(c^{(r)}) = c^{(r+1)}$ at every round. The vector $c^{(0)}$ lies in the initial decoder list, whose cardinality is at most 240. This proves existence of one coherent initial candidate. It does not multiply four independently chosen list sizes because every later candidate is selected as a predecessor on the same chain.

The backward selector sees the complete challenge tuple, so it is not used to justify Fiat–Shamir probabilities. The forward proof supplies that step. At each transcript prefix it chooses a deterministic nearby candidate whenever one exists. The one-round reduction defines the set of challenges that can move this strategy from a prefix with no valid continuation to one with a valid continuation. That set is a function of the current prefix, hence is fixed before its challenge. An accepted execution starting without a close initial candidate must therefore incur the query miss or cross one of these four prefix-determined sets.

For the relation bridge, consider one fibre $v = (v_0, v_1, v_2, v_3)$ with weights $w = (w_0, w_1, w_2, w_3)$. Define

$$F_\alpha(v) = v_0 + \alpha v_1 + \alpha^2 v_2 + \alpha^3 v_3,$$
$$D_\alpha(w) = \frac{w_0 + \alpha^3 w_1 + \alpha^2 w_2 + \alpha w_3}{4}.$$

Expanding $F_\alpha(v) D_\alpha(w)$ by powers of α gives a degree-six polynomial. Its deployed boundary readout $4(p_0 + p_4)$ is exactly $\sum_{i=0}^{3} v_i w_i$, and its evaluation at α is the folded product. Summing over all fibres yields Theorem 3.4. Induction over the four dimensions then carries the initial dot-product claim to the final four coefficients using precisely the challenges already fixed by the transcript.

C Failure-decomposition proof

The detailed protocol ledger contains the events

$$C_{\text{query}}, C_{\text{fold},1}, \ldots, C_{\text{fold},4}, C_{\text{relation}}.$$

Lean defines C as their union. The external ledger contains the eighteen events grouped in Table 5. Duplicate-freedom and disjoint indexing are proved for both lists, followed by the exact set identity

$$C \cup \bigcup_{i=1}^{18} E_i = \bigcup_{k=1}^{24} F_k,$$

where the F_k are the original detailed events. Coverage of every accepted false outcome is proved against the right-hand side and transferred through this identity.

Monotonicity of the probability measure and finite subadditivity then yield Equation (5). Under D_{prod}, twelve deterministic mismatch events are empty. The remaining sum contains SHA-256 collision, target-preimage, and random-oracle terms; Poseidon2 collision and target-preimage terms; and credential recovery.

The numerical endpoint uses exact rational arithmetic. If

$$\Pr[C] \le \frac{7}{10 \cdot 2^{100}} \quad \text{and} \quad \sum_{j=1}^{6} \Pr[P_j] \le \frac{3}{10 \cdot 2^{100}},$$

then

$$\Pr[A_{\mathrm{false}}] \le \frac{10}{10 \cdot 2^{100}} = 2^{-100}.$$

D Published-theorem interface map

Table 9 makes the literature boundary literal. The S-two references are to the 24 March 2026 revision [14]. Here $\alpha = 1 - \theta$ denotes agreement, M is the degree of the challenge-indexed curve, and m is the Guruswami–Sudan multiplicity. The named Aspis predicates are assumptions at the published-theorem boundary; the displayed substitutions and the predicate-to-verifier transports are checked in Lean.

Table 9: Exact external theorem interfaces and V5 substitutions.

Published result	Concrete V5 substitution	Aspis predicate and checked consequence
S-two Protocol 3, Theorem 19 (batching), Appendix A.2 Corollary 1, and Equations (73)–(74)	$F_q = \mathbb{F}_{2^{31}-1}$, $F = K = \mathbb{K}$; $N = 1024$, $\lvert D\rvert = 2^{19}$, $\rho = N/\lvert D\rvert = 1/512$, $\delta = 1 - \rho$, $\alpha = (21/20)\sqrt{\rho}$, $\theta = 1 - \alpha$, $M = 18$, $m = 10$, $\lfloor \alpha \lvert D\rvert \rfloor = 24{,}328$, and $\ell_{\mathrm{GS}} < 240$.	`PublishedAppendixA2 Width19CurveDecodability`; Lean checks the quartic field, strict circle subcode, domain, degree, agreement floor, list cap, Johnson interval, and the $\mathbb{K}^*$ denominator.
S-two Protocol 3, Theorem 19 (folding), Appendix A.2 Theorem 25, and Equations (73)–(74)	Degree $M = 3$; common $\alpha = (21/20)\sqrt{1/512}$. The tuples $(\lvert D\rvert, k-1, \lfloor \alpha \lvert D\rvert \rfloor, m)$ are $(131072, 255, 6082, 10)$, $(32768, 63, 1520, 9)$, $(8192, 15, 380, 6)$, and $(2048, 3, 95, 3)$, with $\rho = (k-1)/\lvert D\rvert$ and $\theta = 1 - \alpha$ in every round.	`PublishedOrdinary PolynomialCurveDecoding`; proved encoder realizations transport it to `V5OutputEncoder CurveDecoding` for the four exact folded domains and bases.
S-two Theorem 22, Equations (89)–(90), instantiating the BCS state-restoration theorem	$R = 30$ post-initial-ensemble public-coin rounds, $\lambda = 256$, and $1 \le Q \le 2^{128}$. With $\varepsilon_{\mathrm{I}} = \max_i \varepsilon_i$, the normalized bound is $(1 + 30/Q)\varepsilon_{\mathrm{I}} + 3(Q + 1/Q)2^{-256}$.	`M0ExcludedBCS DeploymentPremises`; the exact arithmetic function is `bcsError` with $(R, \mathit{capErr}) = (30, 2^{-256})$. The predicate separately requires public-coin semantics, the random-oracle/Fiat–Shamir comparison, and PCS/Merkle applicability.

E Query-sampler calculation

Let $W = (w_0, \ldots, w_{63})$ be the sixty-four masked words obtained from eight SHA-256 blocks, and let $\mathsf{first}_{18}(W)$ keep the first eighteen distinct values. For any ordered distinct tuple $q = (q_0, \ldots, q_{17})$, the set of words satisfying $\mathsf{first}_{18}(W) = q$ has the same cardinality. The proof partitions words by

the draw at which the eighteenth new value first appears, then uses a permutation of the query domain to map the preimages of any tuple to those of any other tuple.

For a bad set B with $|B| = A$, the number of ordered distinct eighteen-tuples contained in B is the falling factorial $(A)_{18}$. The total number of ordered distinct tuples is $(N)_{18}$. Their ratio is

$$\frac{(A)_{18}}{(N)_{18}} = \prod_{j=0}^{17} \frac{A-j}{N-j},$$

which gives Equation (7). Rejection on exhaustion can only decrease the unconditioned bad-query probability.

F Artifact checklist

The artifact guide supplies exact commands, files, declarations, revisions, and digests. A complete independent check has four parts.

1. Run the accepted-execution replay from a clean checkout. Confirm the 331-file closure, pinned translation revision, successful Lean build, and final axiom report.

2. Verify the release checksum manifest. Check the program, proof, statement, signed transaction, and captured RPC response digests.

3. Rebuild the program from the recorded clean source with the pinned Solana and Rust tools, then compare its bytes with the archived binary.

4. Run the offline release verifier and inspect the captured transaction and account records. Confirm slot 435,019,536, the 1,334,452 compute units, the public-statement fields, marker creation, pool update, and later cleanup refund.

The guide also maps every named theorem in this paper to the corresponding Lean declaration. Keeping this map outside the manuscript lets declaration names evolve without turning repository terminology into mathematical prose.

References

[1] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub. Jasmin: High-assurance and high-speed cryptography. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1807–1823. ACM, 2017. doi: 10.1145/3133956.3134078. URL `https://doi.org/10.1145/3133956.3134078`.

[2] José Bacelar Almeida, Manuel Barbosa, Karim Eldefrawy, Stéphane Graham-Lengrand, Hugo Pacheco, and Vitor Pereira. Machine-checked ZKP for NP-relations: Formally verified security proofs and implementations of MPC-in-the-Head. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2587–2600. ACM, 2021. doi: 10.1145/3460120.3484771. URL `https://doi.org/10.1145/3460120.3484771`.

[3] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed–Solomon proximity testing with super-fast verification. In *Advances in Cryptology–EUROCRYPT 2025, Part IV*, volume 15604 of *Lecture Notes in Computer Science*, pages 214–243. Springer, 2025. doi: 10.1007/978-3-031-91134-7_8. URL `https://doi.org/10.1007/978-3-031-91134-7_8`.

[4] Dominic Barker. Aspis: Source, formal development, and evidence archive. Frozen V5 publication artifact, 2026. URL https://github.com/DJBarker87/aspis/tree/aspis-v5-formalization-paper-v1. Immutable publication tag with artifact guide and archived execution evidence; development repository at https://github.com/DJBarker87/aspis.

[5] Gilles Barthe, Juan Manuel Crespo, Benjamin Grégoire, César Kunz, and Santiago Zanella-Béguelin. Computer-aided cryptographic proofs. In *Interactive Theorem Proving*, volume 7406 of *Lecture Notes in Computer Science*, pages 11–27. Springer, 2012. doi: 10.1007/978-3-642-32347-8_2. URL https://doi.org/10.1007/978-3-642-32347-8_2.

[6] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474. IEEE, 2014. doi: 10.1109/SP.2014.36. URL https://doi.org/10.1109/SP.2014.36.

[7] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Theory of Cryptography, TCC 2016-B, Part II*, volume 9986 of *Lecture Notes in Computer Science*, pages 31–60. Springer, 2016. doi: 10.1007/978-3-662-53644-5_2. URL https://doi.org/10.1007/978-3-662-53644-5_2.

[8] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, volume 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 14:1–14:17. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018. doi: 10.4230/LIPIcs.ICALP.2018.14. URL https://doi.org/10.4230/LIPIcs.ICALP.2018.14.

[9] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, 2018. URL https://eprint.iacr.org/2018/046.

[10] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. DEEP-FRI: Sampling outside the box improves soundness. In *11th Innovations in Theoretical Computer Science Conference*, volume 151 of *Leibniz International Proceedings in Informatics*, pages 5:1–5:32. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2020. doi: 10.4230/LIPIcs.ITCS.2020.5. URL https://doi.org/10.4230/LIPIcs.ITCS.2020.5.

[11] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for Reed–Solomon codes. *Journal of the ACM*, 70(5):1–57, 2023. doi: 10.1145/3614423. URL https://doi.org/10.1145/3614423.

[12] Alexander R. Block, Albert Garreta, Jonathan Katz, Justin Thaler, Pratyush Ranjan Tiwari, and Michał Zając. Fiat–Shamir security of FRI and related SNARKs. In *Advances in Cryptology–ASIACRYPT 2023, Part II*, volume 14439 of *Lecture Notes in Computer Science*, pages 3–40. Springer, 2023. doi: 10.1007/978-981-99-8724-5_1. URL https://doi.org/10.1007/978-981-99-8724-5_1.

[13] Sarah Bordage, Alessandro Chiesa, Ziyi Guan, and Ignacio Manzur. All polynomial generators preserve distance with mutual correlated agreement. Cryptology ePrint Archive, Report 2025/2051, 2025. URL https://eprint.iacr.org/2025/2051.

[14] Dan Carmon, Lior Goldberg, Ulrich Haböck, Leonardo Lerer, Ilya Lesokhin, Shahar Papini, and Shahar Samocha. S-two whitepaper. Cryptology ePrint Archive, Report 2026/532, 2026. URL https://eprint.iacr.org/2026/532. Revision dated 24 March 2026.

[15] Elizabeth Crites and Alistair Stewart. On Reed–Solomon proximity gaps conjectures. Cryptology ePrint Archive, Report 2025/2046, 2025. URL https://eprint.iacr.org/2025/2046.

[16] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction–CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37. URL https://doi.org/10.1007/978-3-030-79876-5_37.

[17] Elusiv contributors. The Elusiv SDK: Open-source private sends on solana. Project release announcement, 2023. URL https://medium.com/elusiv-privacy/the-elusiv-sdk-now-open-source-2a5ef7f9d

bb0. Released for Solana mainnet and devnet.

[18] Andres Erbsen, Jade Philipoom, Jason Gross, Robert Sloan, and Adam Chlipala. Simple high-level code for cryptographic arithmetic: With proofs, without compromises. In *2019 IEEE Symposium on Security and Privacy*, pages 1202–1219. IEEE, 2019. doi: 10.1109/SP.2019.00005. URL https://doi.org/10.1109/SP.2019.00005.

[19] Denis Firsov and Benjamin Livshits. The ouroboros of ZK: Why verifying the verifier unlocks longer-term ZK innovation. Cryptology ePrint Archive, Report 2024/768, 2024. URL https://eprint.iacr.org/2024/768.

[20] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version of the Poseidon hash function. Cryptology ePrint Archive, Report 2023/323, 2023. URL https://eprint.iacr.org/2023/323.

[21] Ulrich Haböck. A note on mutual correlated agreement for Reed–Solomon codes. Cryptology ePrint Archive, Report 2025/2110, 2025. URL https://eprint.iacr.org/2025/2110.

[22] Ulrich Haböck, Daniel Lubarov, and Jacqueline Nabaglo. Reed–Solomon codes over the circle group. Cryptology ePrint Archive, Report 2023/824, 2023. URL https://eprint.iacr.org/2023/824.

[23] Ulrich Haböck, David Levit, and Shahar Papini. Circle STARKs. Cryptology ePrint Archive, Report 2024/278, 2024. URL https://eprint.iacr.org/2024/278.

[24] Son Ho and Jonathan Protzenko. Aeneas: Rust verification by functional translation. *Proceedings of the ACM on Programming Languages*, 6(ICFP):711–741, 2022. doi: 10.1145/3547647. URL https://doi.org/10.1145/3547647.

[25] Son Ho, Guillaume Boisseau, Lucas Franceschino, Yoann Prak, Aymeric Fromherz, and Jonathan Protzenko. Charon: An analysis framework for rust. In *Computer Aided Verification*, 2025. URL https://arxiv.org/abs/2410.18042. Tool paper; preprint arXiv:2410.18042.

[26] Light Protocol contributors. Groth16 merkle-tree setup artifacts. Source repository, commit 85243034a19b98952440f5823f1a5ab139e52ca3, 2026. URL https://github.com/Lightprotocol/gnark-mt-setup/tree/85243034a19b98952440f5823f1a5ab139e52ca3. Accessed 24 August 2026.

[27] Simon-Philipp Merz and Àlex Rodríguez García. Skipping class: Algebraic attacks exploiting weak matrices and operation modes of Poseidon2(b). Cryptology ePrint Archive, Report 2026/306, 2026. URL https://eprint.iacr.org/2026/306.

[28] Privacy Cash contributors. Privacy cash mainnet program account. Solana Explorer, 2026. URL https://explorer.solana.com/address/9fhQBbumKEFuXtMBDw8AaQyAjCorLGJQiS3skWZdQyQD. Accessed 24 August 2026.

[29] Privacy Cash contributors. Privacy cash circuit artifacts and trusted-setup record. Source repository, commit aa6818b76b23edcf05b8b9829a9cd900fdb1d241, 2026. URL https://github.com/Privacy-Cash/privacy-cash/tree/aa6818b76b23edcf05b8b9829a9cd900fdb1d241. Accessed 24 August 2026.

[30] Protocol 01 contributors. Protocol 01: Transparent shielded payments on solana. Source repository, commit 790dbff34ac96302fc6336d411fac14f683f5be9, 2026. URL https://github.com/IsSlashy/Protocol-01/tree/790dbff34ac96302fc6336d411fac14f683f5be9. Accessed 24 August 2026.

[31] Solana Foundation. Confidential transfer: Token extensions documentation. Technical documentation, 2026. URL https://solana.com/docs/tokens/extensions/confidential-transfer. Accessed 25 August 2026.

[32] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824. URL https://doi.org/10.1145/3372885.3373824.

[33] Verified-zkEVM contributors. ArkLib: Formally verified arguments of knowledge in lean. Source repository, commit 5a9778db3cafa29288c194077a3780bf524f1137, 2026. URL https://github.com/Verified-zkEVM/ArkLib/tree/5a9778db3cafa29288c194077a3780bf524f1137. Accessed 24 August 2026.

[34] Jian Sheng Wang. HFIPay: Privacy-preserving identifier-routed payments with verifiable claim binding.

arXiv preprint arXiv:2603.26970v2, 2026. URL https://arxiv.org/abs/2603.26970. Revision dated 21 June 2026.

[35] Jotaro Yano. solana-pqzk-fullchain: Full on-chain verification of ZK-STARK proofs and post-quantum signatures on solana. *Journal of Open Source Software*, 11(123):9923, 2026. doi: 10.21105/joss.09923. URL https://doi.org/10.21105/joss.09923.

[36] Jean-Karim Zinzindohoué, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. HACL*: A verified modern cryptographic library. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1789–1806. ACM, 2017. doi: 10.1145/3133956.3134043. URL https://doi.org/10.1145/3133956.3134043.